

RAG系统性能优化技术报告

从召回率到端到端延迟的全链路调优实践

作者：智能检索技术组

日期：2026-07-30

文档编号：RAG-2026-002

目 录

第1章 性能问题定位方法

第2章 检索链路优化

第3章 生成与并发优化

第4章 优化效果与总结

第1章 性能问题定位方法

RAG(检索增强生成)系统的性能瓶颈往往分布在多个环节，仅优化单一节点难以取得整体收益。本报告提出一套全链路性能定位方法，通过分阶段埋点采集召回、重排、上下文组装与生成各环节的耗时与质量指标。

在召回阶段，重点监控向量检索的召回率(Recall@k)与查询延迟；在重排阶段关注精排模型的吞吐与排序质量增益；在生成阶段则跟踪首 token 延迟(TTFT)与整段生成时长。通过火焰图与时序对比快速锁定瓶颈环节。

定位过程中需区分稳态性能与长尾性能，针对 P99 延迟单独分析，避免被平均值掩盖偶发抖动问题。

第2章 检索链路优化

检索链路优化的首要手段是改进切片策略。采用基于语义边界的递归切片，配合重叠窗口，显著减少知识被截断导致的语义损失。同时引入父子切片架构，检索时命中子切片，返回时上溯父切片，兼顾召回精度与上下文完整性。

在向量索引层面，通过 HNSW 参数调优(M 值、efConstruction、efSearch)在召回率与延迟之间取得平衡，并对高频查询结果做多级缓存，命中缓存时直接返回，降低向量库压力。

重排环节引入轻量级交叉编码器对 Top-K 候选精排，虽然增加少量延迟，但可将答案准确率提升两位数百分比，整体投入产出比显著为正。

第3章 生成与并发优化

生成阶段的优化重点在于上下文窗口的合理利用与 prompt 压缩。通过相关性阈值过滤与上下文去重，剔除低贡献片段，使有限窗口聚焦高价值信息，同时减少 token 消耗与生成时长。

在并发处理上，采用流式输出降低用户感知延迟，并对模型推理服务进行批处理(batching)改造，将多个请求的 prefill 阶段合并，提升 GPU 利用率。

对于高频问答场景，可引入语义缓存，对相似问题直接复用历史答案，在保证质量的前提下大幅降低生成负载。

第4章 优化效果与总结

经过全链路调优，P95 端到端延迟从 4.2 秒降至 1.6 秒，答案准确率(人工评估)从 72% 提升至 88%，单位查询成本下降约 35%，达到预期目标。

性能优化是一个持续过程，建议建立常态化性能巡检机制，对关键指标设置阈值告警，防止因数据量增长或模型变更导致性能回退。

本报告总结的方法论与参数基线，可作为后续 RAG 项目性能调优的参考起点，具体参数需结合业务数据特征进一步校准。